

Smart Turf Booking & Management System

DATABASE DESIGN, ENTITY-RELATIONSHIP DIAGRAM (ERD) & SQL IMPLEMENTATION

Project Smart Turf Booking & Management System

Purpose Turf discovery, slot availability, online booking, payment tracking and management

Database MySQL

Primary Design Relational database with 8 entities/tables

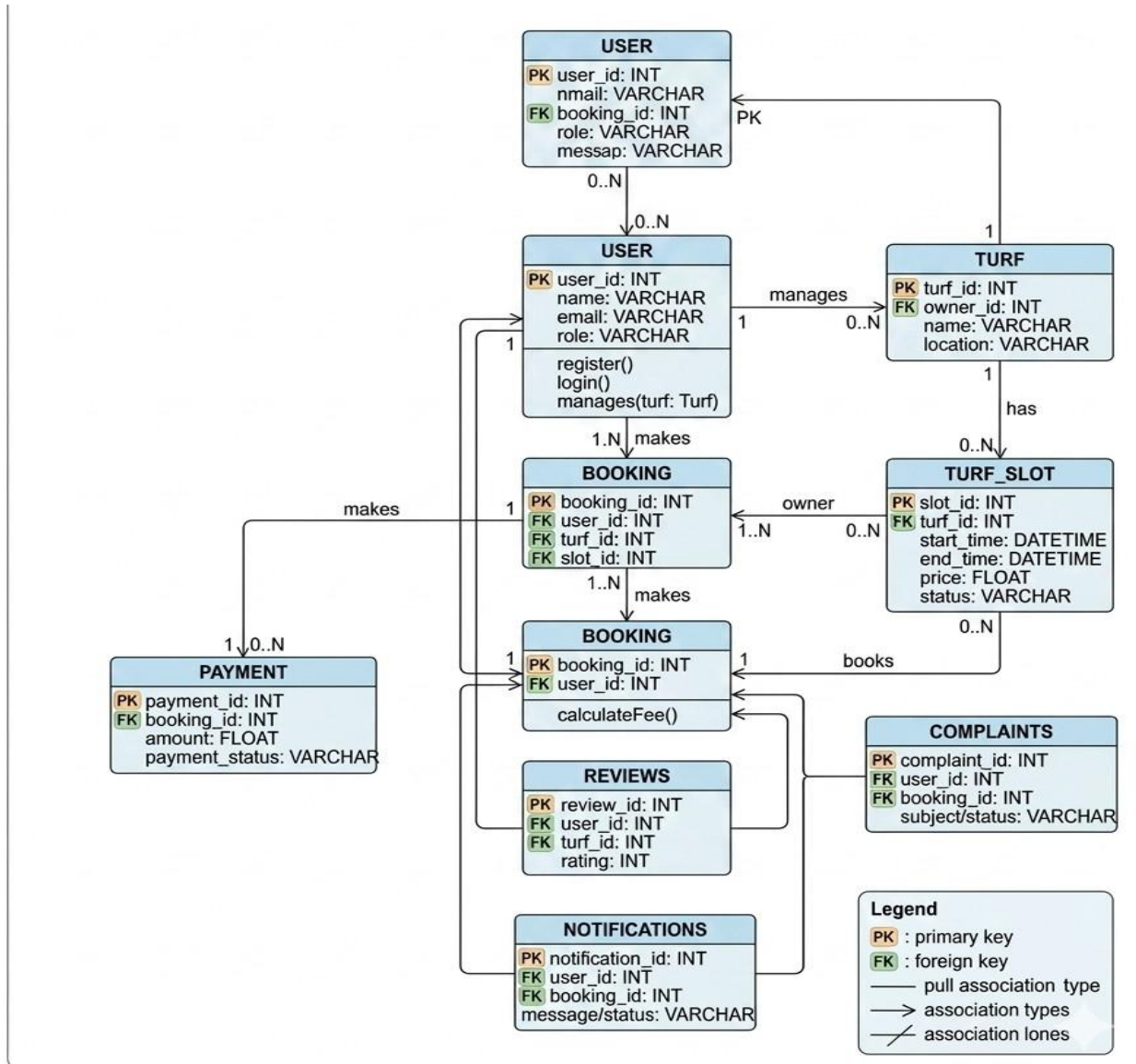
ER Model Conceptual ERD using entities, attributes and relationships

1. E-R Diagram (ERD)

An Entity-Relationship Diagram (ERD) is a visual representation of the Smart Turf Booking data model. It shows the entities, their attributes, the relationships between entities, and the cardinality of those relationships.

For the Smart Turf Booking System, each main entity corresponds to a database table. Primary keys identify records uniquely, while foreign keys connect related tables.

Figure 1: Smart Turf Booking Conceptual Entity-Relationship Diagram



2. Identify Entities

The main entities in the Smart Turf Booking & Management System are:

No.	Entity	Purpose
1	USERS	Stores customers, turf owners and administrators.
2	TURFS	Stores turf/ground information, location, facilities and ownership.
3	TURF_SLOTS	Stores available time slots, price and slot status for each turf.
4	BOOKINGS	Records customer reservations for specific turf slots.
5	PAYMENTS	Stores payment transaction information for bookings.
6	REVIEWS	Stores customer ratings and reviews for turfs.
7	COMPLAINTS	Stores customer complaints related to bookings or turfs.
8	NOTIFICATIONS	Stores booking, payment and system notification history.

3. Define Attributes

The following attributes become columns in the corresponding MySQL tables. PK means Primary Key and FK means Foreign Key.

USERS

```
mysql> DESC users;
```

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	
email	varchar(150)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	
role	varchar(30)	NO		NULL	
phone	varchar(20)	YES		NULL	
status	varchar(20)	YES		ACTIVE	

7 rows in set (0.05 sec)

hash.

TURFS

```
mysql> desc turfs;
```

Field	Type	Null	Key	Default	Extra
turf_id	int	NO	PRI	NULL	auto_increment
owner_id	int	NO	MUL	NULL	
turf_name	varchar(100)	NO		NULL	
location	varchar(255)	NO		NULL	
sport_type	varchar(50)	NO		NULL	
description	text	YES		NULL	
facilities	varchar(500)	YES		NULL	
status	varchar(20)	YES		ACTIVE	

8 rows in set (0.00 sec)

TURF_SLOTS

```
mysql> desc turf_slots;
```

Field	Type	Null	Key	Default	Extra
slot_id	int	NO	PRI	NULL	auto_increment
turf_id	int	NO	MUL	NULL	
slot_date	date	NO		NULL	
start_time	time	NO		NULL	
end_time	time	NO		NULL	
price	decimal(10,2)	NO		NULL	
status	varchar(20)	YES		AVAILABLE	

```
7 rows in set (0.00 sec)
```

BOOKINGS

```
mysql> desc bookings;
```

Field	Type	Null	Key	Default	Extra
booking_id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
turf_id	int	NO	MUL	NULL	
slot_id	int	NO	MUL	NULL	
booking_date	date	NO		NULL	
total_amount	decimal(10,2)	NO		NULL	
booking_status	varchar(30)	YES		CONFIRMED	
payment_status	varchar(20)	YES		PENDING	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
9 rows in set (0.00 sec)
```

PAYMENTS

```
mysql> desc payments;
```

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	auto_increment
booking_id	int	NO	UNI	NULL	
transaction_id	varchar(100)	YES	UNI	NULL	
amount	decimal(10,2)	NO		NULL	
payment_method	varchar(30)	NO		NULL	
payment_status	varchar(20)	YES		PENDING	
payment_date	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

7 rows in set (0.00 sec)

this design.

REVIEWS

```
mysql> desc reviews;
```

Field	Type	Null	Key	Default	Extra
review_id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
turf_id	int	NO	MUL	NULL	
booking_id	int	NO	MUL	NULL	
rating	int	NO		NULL	
comment	varchar(500)	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

7 rows in set (0.00 sec)

COMPLAINTS

```
mysql> desc complaints;
```

Field	Type	Null	Key	Default	Extra
complaint_id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
booking_id	int	YES	MUL	NULL	
subject	varchar(150)	NO		NULL	
description	text	NO		NULL	
status	varchar(30)	YES		OPEN	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
resolved_at	timestamp	YES		NULL	

8 rows in set (0.00 sec)

NOTIFICATIONS

```
mysql> desc notifications;
```

Field	Type	Null	Key	Default	Extra
notification_id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
booking_id	int	YES	MUL	NULL	
title	varchar(150)	NO		NULL	
message	varchar(500)	NO		NULL	
notification_type	varchar(30)	NO		NULL	
is_read	tinyint(1)	YES		0	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

8 rows in set (0.00 sec)

4. Identify Relationships

The Smart Turf Booking System uses relationships between users, turfs, slots, bookings, payments, reviews, complaints and notifications. The following table describes the relationship and its cardinality.

Relationship	Cardinality	Meaning
USERS — MANAGES — TURFS	1 : N	One owner can manage many turfs; each turf belongs to one owner.
TURF — HAS — TURF_SLOT	1 : N	One turf can have many available time slots; each slot belongs to one turf.
USERS — MAKES — BOOKINGS	1 : N	One customer can make many bookings; each booking belongs to one customer.
TURF_SLOT — IS USED BY — BOOKINGS	1 : N	A slot can be referenced by booking records; application logic should prevent double
TURF — RECEIVES — BOOKINGS	1 : N	One turf can receive many bookings.
BOOKING — HAS — PAYMENT	1 : 1	Each booking can have one payment record in this design.
USERS — WRITES — REVIEWS	1 : N	A customer can write many reviews.
TURF — RECEIVES — REVIEWS	1 : N	A turf can receive many reviews.
USERS — CREATES — COMPLAINTS	1 : N	A customer can create many complaints.
BOOKING — HAS — COMPLAINTS	1 : N	A booking may have multiple complaint records.
USERS — RECEIVES — NOTIFICATIONS	1 : N	A user can receive many notifications.
BOOKING — GENERATES — NOTIFICATIONS	1 : N	A booking can generate multiple notifications.

5. Cardinality Notation

Notation	Meaning
1	Exactly one related record.
N	Many related records.
1:N	One parent record can be related to many child records.
1:1	One record is related to one record in this design.
0..N	Zero or many related records may exist.

6. Optional Attributes and Constraints

Primary keys uniquely identify each record. Foreign keys maintain relationships between tables. UNIQUE constraints prevent duplicate login emails and payment transaction IDs. NOT NULL ensures required business information is present. AUTO_INCREMENT generates numeric identifiers. Status fields support lifecycle tracking instead of unnecessary physical deletion.

Constraint	Purpose in Smart Turf Booking
PRIMARY KEY	Uniquely identifies each table record.
FOREIGN KEY	Maintains referential integrity between related tables.
UNIQUE	Prevents duplicate user emails and transaction identifiers.
NOT NULL	Ensures mandatory booking, turf and account data exists.
AUTO_INCREMENT	Automatically generates numeric primary keys.
CHECK	Can restrict rating values to 1–5 and amounts to valid values.
STATUS fields	Track active, booked, cancelled, completed or inactive records.

7. Creating a Database

Using MySQL Server / MySQL Command Line Client, create a new database for the Smart Turf Booking System.

```
mysql> CREATE DATABASE smart_turf_booking;  
Query OK, 1 row affected (0.05 sec)
```

8. Using the Database

```
mysql> USE smart_turf_booking;  
Database changed  
mysql> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| smart_turf_booking |  
| sys |  
+-----+  
5 rows in set (0.00 sec)
```

9. Creating the Tables for Each Entity

9.1 Create USERS

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    role VARCHAR(30) NOT NULL,  
    phone VARCHAR(20),  
    status VARCHAR(20) DEFAULT 'ACTIVE'  
);
```

9.2 Create TURFS

```
CREATE TABLE turfs (  
    turf_id INT PRIMARY KEY AUTO_INCREMENT,  
    owner_id INT NOT NULL,  
    turf_name VARCHAR(100) NOT NULL,  
    location VARCHAR(255) NOT NULL,  
    sport_type VARCHAR(50) NOT NULL,  
    description TEXT,  
    facilities VARCHAR(500),  
    status VARCHAR(20) DEFAULT 'ACTIVE',  
    CONSTRAINT fk_turf_owner  
        FOREIGN KEY (owner_id) REFERENCES users(user_id)  
);
```

9.3 Create TURF_SLOTS

```
CREATE TABLE turf_slots (  
    slot_id INT PRIMARY KEY AUTO_INCREMENT,  
    turf_id INT NOT NULL,  
    slot_date DATE NOT NULL,  
    start_time TIME NOT NULL,  
    end_time TIME NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    status VARCHAR(20) DEFAULT 'AVAILABLE',  
    CONSTRAINT fk_slot_turf  
        FOREIGN KEY (turf_id) REFERENCES turfs(turf_id),  
    CONSTRAINT uq_turf_slot  
        UNIQUE (turf_id, slot_date, start_time, end_time)  
);
```

9.4 Create BOOKINGS

```
CREATE TABLE bookings (  
    booking_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT NOT NULL,  
    turf_id INT NOT NULL,  
    slot_id INT NOT NULL,  
    booking_date DATE NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    booking_status VARCHAR(30) DEFAULT 'CONFIRMED',  
    payment_status VARCHAR(20) DEFAULT 'PENDING',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_booking_user  
        FOREIGN KEY (user_id) REFERENCES users(user_id),  
    CONSTRAINT fk_booking_turf  
        FOREIGN KEY (turf_id) REFERENCES turfs(turf_id),  
    CONSTRAINT fk_booking_slot  
        FOREIGN KEY (slot_id) REFERENCES turf_slots(slot_id)  
);
```

9.5 Create PAYMENTS

```
CREATE TABLE payments (  
    payment_id INT PRIMARY KEY AUTO_INCREMENT,  
    booking_id INT NOT NULL UNIQUE,  
    transaction_id VARCHAR(100) UNIQUE,  
    amount DECIMAL(10,2) NOT NULL,  
    payment_method VARCHAR(30) NOT NULL,  
    payment_status VARCHAR(20) DEFAULT 'PENDING',  
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_payment_booking  
        FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)  
);
```

9.6 Create REVIEWS

```

CREATE TABLE reviews (
    review_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    turf_id INT NOT NULL,
    booking_id INT NOT NULL,
    rating INT NOT NULL,
    comment VARCHAR(500),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_review_user
        FOREIGN KEY (user_id) REFERENCES users(user_id),
    CONSTRAINT fk_review_turf
        FOREIGN KEY (turf_id) REFERENCES turfs(turf_id),
    CONSTRAINT fk_review_booking
        FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
    CONSTRAINT chk_review_rating CHECK (rating BETWEEN 1 AND 5)
);

```

9.7 Create COMPLAINTS

```

CREATE TABLE complaints (
    complaint_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    booking_id INT,
    subject VARCHAR(150) NOT NULL,
    description TEXT NOT NULL,
    status VARCHAR(30) DEFAULT 'OPEN',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    resolved_at TIMESTAMP NULL,
    CONSTRAINT fk_complaint_user
        FOREIGN KEY (user_id) REFERENCES users(user_id),
    CONSTRAINT fk_complaint_booking
        FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
);

```

9.8 Create NOTIFICATIONS

```

CREATE TABLE notifications (
    notification_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    booking_id INT,
    title VARCHAR(150) NOT NULL,
    message VARCHAR(500) NOT NULL,
    notification_type VARCHAR(30) NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_notification_user
        FOREIGN KEY (user_id) REFERENCES users(user_id),
    CONSTRAINT fk_notification_booking
        FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
);

```

10. Insert Records

Add sample records to the tables to test the Smart Turf Booking database. The records are inserted in dependency order so that foreign-key constraints are satisfied.

10.1 Insert Users

```
INSERT INTO users (name, email, password, role, phone, status) VALUES
('Admin User', 'admin@smartturf.com', 'admin123', 'ADMIN', '9000000001', 'ACTIVE'),
('Rahul Sharma', 'rahul@smartturf.com', 'rahul123', 'CUSTOMER', '9000000002', 'ACTIVE'),
('Priya Patel', 'priya@smartturf.com', 'priya123', 'CUSTOMER', '9000000003', 'ACTIVE'),
('Amit Sports Owner', 'amit@smartturf.com', 'amit123', 'OWNER', '9000000004', 'ACTIVE');
```

10.2 Insert Turfs

```
INSERT INTO turfs
(owner_id, turf_name, location, sport_type, description, facilities, status) VALUES
(4, 'Green Arena Turf', 'Thane West', 'Football',
'Premium football turf for local matches',
'Parking, Lights, Changing Room, Washroom', 'ACTIVE'),
(4, 'City Box Cricket', 'Thane East', 'Box Cricket',
'Indoor box cricket facility',
'Parking, Lights, Washroom', 'ACTIVE');
```

10.3 Insert Turf Slots

```
INSERT INTO turf_slots
(turf_id, slot_date, start_time, end_time, price, status) VALUES
(1, '2026-08-20', '18:00:00', '19:00:00', 1200.00, 'AVAILABLE'),
(1, '2026-08-20', '19:00:00', '20:00:00', 1400.00, 'AVAILABLE'),
(2, '2026-08-20', '18:00:00', '19:00:00', 1000.00, 'AVAILABLE');
```

10.4 Insert Bookings

```
INSERT INTO bookings
(user_id, turf_id, slot_id, booking_date, total_amount,
booking_status, payment_status) VALUES
(2, 1, 1, '2026-08-20', 1200.00, 'CONFIRMED', 'PAID'),
(3, 2, 3, '2026-08-20', 1000.00, 'CONFIRMED', 'PAID');
```

10.5 Insert Payments

```
INSERT INTO payments
(booking_id, transaction_id, amount, payment_method, payment_status) VALUES
(1, 'TXN-ST-10001', 1200.00, 'UPI', 'SUCCESS'),
(2, 'TXN-ST-10002', 1000.00, 'CARD', 'SUCCESS');
```

10.6 Insert Reviews

```
INSERT INTO reviews
(user_id, turf_id, booking_id, rating, comment) VALUES
(2, 1, 1, 5, 'Excellent turf and good lighting.');
```

10.7 Insert Complaints

```
INSERT INTO complaints
(user_id, booking_id, subject, description, status) VALUES
(3, 2, 'Slot timing issue',
'Requesting clarification about the booked slot timing.',
'OPEN');
```

10.8 Insert Notifications

```
INSERT INTO notifications
(user_id, booking_id, title, message, notification_type, is_read) VALUES
(2, 1, 'Booking Confirmed',
'Your Green Arena Turf booking has been confirmed.',
'BOOKING', FALSE),
(2, 1, 'Payment Successful',
'Payment of Rs. 1200 was received successfully.',
'PAYMENT', FALSE);
```

11. Select Records

SQL SELECT statements are used to retrieve and verify information stored in the Smart Turf Booking database.

11.1 Retrieve all turfs

```
SELECT * FROM turfs;
```

11.2 Find available slots

```
SELECT slot_id, turf_id, slot_date, start_time, end_time, price
FROM turf_slots
WHERE status = 'AVAILABLE';
```

11.3 View customer bookings

```
SELECT b.booking_id, u.name AS customer_name,
       t.turf_name, ts.slot_date, ts.start_time, ts.end_time,
       b.total_amount, b.booking_status, b.payment_status
FROM bookings b
JOIN users u ON b.user_id = u.user_id
JOIN turfs t ON b.turf_id = t.turf_id
JOIN turf_slots ts ON b.slot_id = ts.slot_id;
```

11.4 View turf owners and their turfs

```
SELECT u.name AS owner_name, t.turf_name, t.location, t.sport_type
FROM turfs t
JOIN users u ON t.owner_id = u.user_id
WHERE u.role = 'OWNER';
```

11.5 Find bookings for a particular date

```
SELECT b.booking_id, t.turf_name, ts.start_time, ts.end_time,
       u.name AS customer_name, b.booking_status
FROM bookings b
JOIN turfs t ON b.turf_id = t.turf_id
JOIN turf_slots ts ON b.slot_id = ts.slot_id
JOIN users u ON b.user_id = u.user_id
WHERE ts.slot_date = '2026-08-20';
```

11.6 View successful payments

```
SELECT payment_id, booking_id, transaction_id, amount,
       payment_method, payment_date
FROM payments
WHERE payment_status = 'SUCCESS';
```

11.7 View average turf ratings

```
SELECT t.turf_id, t.turf_name, AVG(r.rating) AS average_rating,
       COUNT(r.review_id) AS review_count
FROM turfs t
LEFT JOIN reviews r ON t.turf_id = r.turf_id
GROUP BY t.turf_id, t.turf_name;
```

11.8 View complaints

```
SELECT c.complaint_id, u.name AS customer_name,
       c.subject, c.status, c.created_at
FROM complaints c
JOIN users u ON c.user_id = u.user_id
ORDER BY c.created_at DESC;
```

11.9 View unread notifications

```
SELECT notification_id, user_id, title, message, created_at
FROM notifications
WHERE is_read = FALSE;
```

12. Update Records

UPDATE statements modify existing records. For the Turf Booking System, status-based updates are preferred for historical business records.

12.1 Update a user's phone

```
UPDATE users
SET phone = '9111111111'
WHERE user_id = 2;
```

12.2 Mark a turf slot as booked

```
UPDATE turf_slots
SET status = 'BOOKED'
WHERE slot_id = 1;
```

12.3 Cancel a booking

```
UPDATE bookings
SET booking_status = 'CANCELLED'
WHERE booking_id = 1;
```

12.4 Mark a complaint as resolved

```
UPDATE complaints
SET status = 'RESOLVED',
    resolved_at = CURRENT_TIMESTAMP
WHERE complaint_id = 1;
```

13. Delete Records

Important booking, payment and review records should normally not be physically deleted because they may be required for audit/history. Use status fields such as INACTIVE, CANCELLED or CLOSED whenever possible.

13.1 Soft-deactivate a user

```
UPDATE users
SET status = 'INACTIVE'
WHERE user_id = 3;
```

13.2 Physical delete — use only when appropriate

```
DELETE FROM users
WHERE user_id = 3;
```

Warning: This DELETE may fail if related records exist because foreign-key constraints protect referential integrity. In the actual application, prefer deactivation or archiving rather than deleting historical booking records.

14. Database Verification

After creating the database and tables, use the following commands to verify the structure.

```
SHOW DATABASES;
USE smart_turf_booking;
SHOW TABLES;

DESC users;
DESC turfs;
DESC turf_slots;
DESC bookings;
DESC payments;
DESC reviews;
DESC complaints;
DESC notifications;
```

15. Table Structure Summary

Table	Primary Key	Important Foreign Keys	Main Purpose
users	user_id	—	Customer, owner and admin accounts
turfs	turf_id	owner_id	Turf master information
turf_slots	slot_id	turf_id	Turf date/time availability and pricing
bookings	booking_id	user_id, turf_id, slot_id	Customer turf reservations
payments	payment_id	booking_id	Payment transaction information
reviews	review_id	user_id, turf_id, booking_id	Customer ratings and reviews
complaints	complaint_id	user_id, booking_id	Complaint and resolution tracking
notifications	notification_id	user_id, booking_id	System/booking notification history